NATIONAL UNIVERSITY OF OSTROH ACADEMY
Department of Information Technologies and Data Analytics

Lab 8.
JavaScript fundamentals: data types and coercion, scope, closures, functions, arrays, objects, and ES modules

Modern Web Development: HTML, CSS, JS/TS

Deadline: 09.10.2026

1. Aim

Understand where JavaScript behaves differently from languages you already know: type coercion, variable scopes and hoisting, closures, values and references, and ES modules. Learn to verify your own intuition and an AI agent's confident answers by running the code, rather than taking them on trust.

Expected outcome: five pure functions written by you that pass the provided tests, and for each function — an explanation of which language trap it avoids and why.

2. Objectives and learning outcomes

After completing this lab, you must be able to:

- Explain the difference between primitives and objects, the results of `typeof`, and why `0.1 + 0.2 !== 0.3`.
- Predict implicit type coercion in `+`, `-`, `==`, `>=`, and inside conditions; choose deliberately between `==` and `===`, `||` and `??`; use `?.`.
- Explain the difference between `var`, `let`, and `const`, block and function scope, hoisting, and the temporal dead zone (TDZ).
- Explain what a closure "remembers" and use it for private state.
- Distinguish between copying a value and copying a reference, shallow and deep copies, array methods that mutate the array, and those that return a new one.
- Use default parameters, rest parameters, and destructuring; explain why a default parameter value does not trigger for `null`.

- Explain how an ES module differs from a classic script.
- Verify pure functions with `node --test` tests and read `assert` failure messages.
- Use an AI agent as a dialogue partner whose answers are verified by running the code, rather than as a code author.

3. Lab tasks

1. Work through the research tasks (§4) and run the experiments: for each one, record your own prediction and the AI agent's prediction **before** running the code.
2. Download the JS Basics starter project, commit it to Git unchanged, and verify that the tests fail (§5.1).
3. Implement the functions for Challenges 1–5 so that all tests pass (§5.3–5.7) — with one commit per challenge.
4. Publish the project on GitHub Pages or Vercel, prepare your report (§6), and defend your work (§7).

> **Important!** This lab has no DOM, events, timers, or Promises — those are topics for Lab 9 and Lecture 6. The functions in `src/` are pure: they receive data via parameters, return a result, and mutate nothing outside themselves.

4. Research tasks

Before starting the practical part, study the sources and run the experiments. General reference — JavaScript Guide — MDN; for deeper reading — You Don't Know JS Yet and the ECMAScript 2025 Language Specification.

Experiment protocol:

1. Record **your own prediction** of the result.
2. Ask the AI agent for its prediction, **without showing** it the result and without allowing it to run the code.
3. Run the code in the browser Console (DevTools). Paste each snippet as a whole.
4. If either prediction did not match the actual result, explain what was predicted incorrectly and why.

Compile the results into a table — it is part of your report:

#	Code	My prediction	AI prediction (without running)	Actual result	Explanation
1	`'5' + 1`	…	…	…	…

Next to each experiment, the problem this behavior leads to in real code is noted. In your explanation, explain how to avoid that problem.

4.1. Data types and type coercion

Study:

- JavaScript data types and data structures — MDN
- Equality comparisons and sameness — MDN
- Type coercion and Falsy — MDN Glossary
- Nullish coalescing operator (??) and Optional chaining (?.) — MDN
- IsLooselyEqual — ECMAScript 2025 — the == algorithm in the specification

Investigate: which types JavaScript has and what `typeof` returns for each, especially for `null`, arrays, and functions; which values are falsy; when `+` concatenates strings and when it adds numbers; which rules govern `==` and why `===` is the standard choice; how `??` differs from `||`; what `?.` returns if the chain breaks; why `NaN` is not equal to itself.

Experiments:

1. `'5' + 1` and `'5' - 1`
   Problem: a form field value is always a string, so `"2" + 1` produces `"21"`.
2. `0.1 + 0.2 === 0.3`
   Problem: sums and averages do not match on the last decimal place.
3. `[NaN === NaN, Number.isNaN(NaN), isNaN('abc'), Number.isNaN('abc')]`
   Problem: checking `x === NaN` never evaluates to true.
4. `[0 || 'none', 0 ?? 'none']`
   Problem: a score or price of 0 is displayed as "no data".
5. `[null == undefined, null == 0, null >= 0]`
   Problem: `null` behaves like 0 in one comparison, but not in another.
6. `[10, 9, 1].sort()`
   Problem: an array of numbers is sorted alphabetically as strings.

4.2. Scope, hoisting, and closures

Study:

- Grammar and types — MDN, Declarations section
- let — MDN, Temporal dead zone section
- Hoisting — MDN Glossary
- Closures — MDN
- You Don't Know JS Yet: Scope & Closures

Investigate: how block scope differs from function scope; what exactly is hoisted for `var`, `let`/`const`, and function declarations; what the TDZ is; why `const` does not make an object immutable; what a closure is and what it retains — the value of a variable at creation time or the variable reference itself; how many separate variables are created by `for (let i …)` versus `for (var i …)`.

Experiments:

7. `console.log(early); var early = 1;`, and then separately `{ console.log(late); let late = 1; }` — inside a block, because top-level `let` in the Console is handled specially.
   Problem: with `var`, code silently works with `undefined` instead of failing at the actual point of error.
8. A loop that creates functions:

   ```
   const viaVar = [];
   for (var i = 0; i < 3; i++) viaVar.push(() => i);
   const viaLet = [];
   for (let j = 0; j < 3; j++) viaLet.push(() => j);
   [viaVar.map((f) => f()), viaLet.map((f) => f())];
   ```

   Problem: all handlers created in the loop see the final value of the counter.
9. `const config = { theme: 'light' }; config.theme = 'dark'; config;`, and then `config = {};`
   Problem: `const` is mistaken for immutability, while the object can be mutated from anywhere that holds a reference to it.

4.3. Values and references: objects and arrays

Study:

- Working with objects — MDN
- Shallow copy and Deep copy — MDN Glossary
- structuredClone() — MDN
- Object.freeze() — MDN
- Array: copying methods and mutating methods — MDN
- Map — MDN

Investigate: what the assignment `b = a` copies for a primitive versus an object; what `===` compares for two objects; what the spread operator (`{ ...obj }`, `[...arr]`) copies and what it does not; when `structuredClone` is needed; what `Object.freeze` does and what happens when attempting to mutate a frozen object in strict mode versus sloppy mode; which array methods mutate the array in place (`sort`, `reverse`, `splice`, `push`) and which return a new one (`toSorted`, `toReversed`, `map`, `filter`, `slice`); how `Map` differs from a plain object as a key-value store and what can be used as its keys.

Experiments:

10. Shallow copy:

    ```
    const user = { name: 'Ann', address: { city: 'Kyiv' } };
    const copy = { ...user };
    copy.name = 'Bob';
    copy.address.city = 'Lviv';
    [user.name, user.address.city];
    ```

    Problem: editing a "copy" in a form modifies the original object because nested objects are shared by reference.

11. Freezing:

    ```
    const defaults = Object.freeze({ theme: 'light', limits: { perPage: 20 } });
    defaults.theme = 'dark';
    defaults.limits.perPage = 100;
    defaults;
    ```

    Then repeat the snippet using the name `strictDefaults` instead of `defaults` and with the `'use strict';` directive at the beginning.
    Problem: "frozen" settings can still be modified through nested objects. Modules always run in strict mode — that is how the starter project's tests catch functions mutating their arguments.

12. `const ratings = [8.2, 9.1, 7.5]; const sorted = ratings.sort(); [sorted === ratings, ratings];`, and then `const scores = [8.2, 9.1, 7.5]; const ranked = scores.toSorted(); [ranked === scores, scores];`
    Problem: an array is sorted "for display", mutating the original array used by other parts of the application.

4.4. Functions and ES modules

Study:

- Functions — MDN
- Arrow function expressions — MDN
- Default parameters, Rest parameters, Destructuring — MDN
- JavaScript modules — MDN, especially Other differences between modules and classic scripts
- Strict mode — MDN

Investigate: how function declarations, function expressions, and arrow functions differ; when a default parameter value triggers; how rest parameters and object destructuring with default values work; how named exports differ from default exports; how an ES module differs from a classic `<script>`: strict mode, its own top-level lexical scope, `this` at the top level, and loading only over HTTP(S).

Experiments:

13. `function greet(name = 'guest') { return name; } [greet(), greet(undefined), greet(null)];`
    Problem: an API returns `null` for a missing field, and the default value does not trigger.
14. Calling before declaration:

    ```
    {
      console.log(declared());
      console.log(arrow());
      function declared() { return 'declaration'; }
      const arrow = () => 'arrow';
    }
    ```

    Problem: a function is moved to the end of a module and rewritten as an arrow function — and earlier code that calls it throws a `ReferenceError`.
15. On the starter project page opened via Live Preview (§5.1), temporarily add to the end of `main.js` the lines `var fromModule = 'module';` and `console.log('this:', this);`, reload the page, and run in the Console `var fromConsole = 'console'; [window.fromModule, window.fromConsole];`. Afterwards, remove those temporary lines from `main.js`.
    Problem: top-level variables of classic scripts pollute `window`, allowing one script to overwrite another's variables.

5. Practical part: Code challenges

The **JS Basics** starter project consists of five modules in `src/` with function stubs, test suites in `test/`, and 30 shows from TVmaze in `data/shows.json`. What each function must do is described in the assignment brief; tests check these exact rules, and each test's name is the rule it verifies. Complete the challenges in order: C2–C4 operate on the output of C1.

5.1. Setup

1. Download js-basics-starter.zip and extract it into a **separate repository** or into a `lab-8/` folder in your shared coursework repository. Make your first commit **unchanged** — all diffs during oral defense are reviewed against it.
2. In the project root, run `node --test`. All tests should fail with `Error: Not implemented`. No installation is needed: `node:test` and `node:assert` are built into Node.js.
3. Open `index.html` with a double-click — the address starts with `file://`. The report will not display, and the Console will show the error: `Access to script at 'file:///…/main.js' from origin 'null' has been blocked by CORS policy`.

Explain in your report why the browser refuses to load a module residing in the same directory.

4. Open the same page via Live Preview (or `npx serve .`). Now the report renders, but shows errors instead of results in each section: the functions are not yet implemented. Refresh the page after completing each challenge.

5.2. Rules for working with an AI agent

- **Research tasks (§4).** The AI gives its prediction **before** running the code. Record its answer in the table exactly as given, without modifying it after testing.
- **Challenges.** The AI may explain concepts, test failure messages, or errors, but **must not write** the function — neither in full nor as a "hint" with ready code. If using an AI agent inside your IDE, use non-editing mode (ask / plan).

Example prompt:

> Explain why this test is failing. Below are its name, the assert message, and my function. Do not write the fixed code: identify the JavaScript rule I likely overlooked, and suggest how to verify it in the Console.

During oral defense, you will modify a function live, without AI (§7). In your report, state the name and version of the agent, model, and date of work.

5.3. Challenge 1 — `normalizeShow`

Transform a TVmaze show record into a flat object used by the remaining functions. The incoming data is incomplete: records contain `null` and missing fields. Topics: §4.1, §4.3. Tests: `test/normalize.test.js`.

5.4. Challenge 2 — `filterShows`

Filter by partial name query, genre, and minimum rating; any filter option may be omitted. Topics: §4.1, §4.4. Tests: `test/filter.test.js`.

5.5. Challenge 3 — `sortShows`

Sort by name, year, or rating in both directions, without mutating the input array. Topics: §4.1, §4.3. Tests: `test/sort.test.js`.

5.6. Challenge 4 — `genreStats`

Calculate show count and average rating for each genre. Topics: §4.1. Tests: `test/stats.test.js`.

5.7. Challenge 5 — `createCounter`, `once`, `memoize`

Three functions that maintain state inside a closure. Topics: §4.2. Tests: `test/closures.test.js`.

5.8. Commits

Each challenge requires a **separate commit** as soon as its tests pass, for example `feat(C3): implement sortShows`. Files in `test/` must not be changed in any commit; temporary lines from Experiment 15 must not be committed. After C5, publish the project to GitHub Pages or Vercel (assignment brief, §6).

6. Report requirements

Email subject:

```
[Web] Lab 8: JavaScript fundamentals: data types and coercion, scope, closures, functions, arrays, objects, and ES modules
```

In the email, include links to your repository and the published page — choose either GitHub Pages or Vercel:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

The report must contain:

1. the table of experiments from §4 (all 15) and brief conclusions for each subsection: where intuition from other languages failed, and where the AI erred;
2. the name and version of the AI agent, model, and date of work;
3. the explanation from §5.1: why `index.html` fails over `file://` and works via Live Preview;
4. the `node --test` output after the final commit — all tests passing;
5. for each challenge: the trap it contained (which test failed on your first implementation, or if none did, what your code guards against), the underlying JavaScript rule, and why your solution works;
6. summary: in which experiments the AI was mistaken and why, in your opinion.

7. Oral defense

Defense involves an oral interview discussing your practical implementation. Functions must be written by you: AI was allowed to explain, but not to write code. You will modify code live during defense without AI and answer questions while sharing your screen.

Sample questions:

Challenges

- Why was test X failing before your fix? Show the corresponding commit.
- Modify a function live: for example, place `null` at the beginning instead of the end, or sort names in descending order. Which tests will be affected and why?
- Why does a test fail with `TypeError: Cannot assign to read only property`?

Types and coercion

- How does `??` differ from `||`, and `==` from `===`? Why is `null >= 0` `true`, while `null == 0` is `false`?
- Why is `0.1 + 0.2 !== 0.3` and how does your C4 account for this?

Scope and closures

- Predict without running the loop of closures using `var`, then using `let`. Explain the difference.
- What does `createCounter` remember, and why do two counter instances not interfere with each other?

References and modules

- What does the spread operator copy, and what does it not? When is `structuredClone` necessary?
- Why does a default parameter value not trigger for `null`?
- Why does `index.html` fail over `file://`, but work via Live Preview? How does an ES module differ from a classic script?

AI

- In which experiment did the AI make a mistake? Why, in your opinion?

8. Grading

This assignment is worth **3 points** (1 point each for research tasks, practical implementation, and defense), plus **1 bonus point**.

Component	Points	Grading criteria
Research tasks (§4)	1	0.25 points for each subsection §4.1–§4.4: all experiments recorded in the table with both predictions and an explanation of how to avoid the problem, plus a subsection conclusion (report item 1); §4.4 also includes the `file://` explanation from §5.1 (item 3)
Practical part (§5)	1	Tests: `node --test` passes completely after the final commit — 0.5; passes completely for at least three challenges — 0.25. Commits and report: separate commit for each challenge, published live page, and documented trap for each challenge (item 5) — 0.5; missing one of the three — 0.25

Component	Points	Grading criteria
Defense (§7)	1	Live modification of your function without AI and explaining which tests it affects — up to 0.5; two questions from other groups in §7 — 0.25 for each complete answer
Bonus point	+1	Either submitted before **09.10.2026** OR completed during class

Tests run against the original `test/` files from the starter project: if modified, original files are used for grading.

> **Important!** Without defense, the research and practical parts combined are capped at **1 point**: defense confirms that you understand the code you wrote, rather than having simply generated results with AI. The bonus point is awarded even without defense.